

Security Risk Brief — Acme Financial Services

To: Engineering Manager | **From:** Security Consulting Engagement, Day 1 | **Purpose:** Before we build anything, here's where the real risk sits today and what to fix first.

1. Passwords and API keys are stored directly in the code repository [CRITICAL]

Right now, .env files containing database passwords and third-party API keys are committed to Git. Anyone with access to the repository — including former employees whose laptops still have a local copy, or anyone who ever forks or clones it — can read these credentials in plain text, with no extra effort.

What happens if this is exploited: Someone with a copy of the repo pulls the database password straight out of the commit history and logs into production. No hacking skill required, no alarms triggered — it looks like a normal login. They could pull customer financial records, alter data, or use a connected API key to run up costs or act as if they were Acme itself. For a financial services company, this is a direct path to a customer data breach and regulatory investigation.

2. Every system has full administrator access to the entire cloud account [CRITICAL]

Every service we run in AWS has been given "Administrator" level permissions — the same access an IT administrator would have to everything in the company's cloud environment, not just the parts that service actually needs.

What happens if this is exploited: If an attacker compromises even one small, low-value service — say, an internal reporting tool with a known bug — they don't just get that tool. They get the keys to everything: every database, every server, every stored file, and the ability to destroy backups to cover their tracks. A minor break-in becomes a company-wide catastrophe because there's no containment between systems.

3. Code goes live with no one else reviewing it first [HIGH]

Developers can currently push changes directly to the live product with no second person checking the work and no automated safety checks along the way.

What happens if this is exploited: A mistake — or a deliberate change by a compromised developer account — goes straight into production. This could be something as damaging as a change that lets a transaction bypass a spending limit, or a hidden backdoor added by an attacker who has stolen a developer's login. Because nobody reviews it, it isn't caught until a customer or auditor finds it.

4. We don't know what software components we're running, or which have known holes in them [HIGH]

The product is built using hundreds of third-party open-source components. We currently have no list of what those are and no process to check whether any of them have publicly known security holes.

What happens if this is exploited: Security researchers publish detailed writeups of vulnerabilities in popular open-source packages every week, often with ready-made attack scripts. If one of our packages is on that list — and given we've never checked, it's likely at least one is — an attacker doesn't need to find a new weakness themselves. They just run a public, pre-built attack against us. This is one of the most common ways companies get breached, precisely because it requires no special skill.

5. The website will accept requests from any other website on the internet [MEDIUM]

A browser security setting that normally restricts which websites can talk to our systems on a user's behalf has been left fully open (the "permissive CORS" item in the current-state summary).

Why this matters: A malicious website a customer happens to visit — even in an unrelated browser tab — could quietly make requests to our systems using that customer's logged-in session, without their knowledge. Rated Medium rather than Critical/High because it requires a user to be actively logged in at the same time, but it's a real, low-effort attack path worth closing.

What to fix first, and why

Start with **#1 and #2 together: get the credentials out of the code and rotate every one of them, and cut down every system's cloud permissions to only what it actually needs.**

The reason these come before everything else: items 3, 4, and 5 are about *preventing the next incident*. Items 1 and 2 are different — they describe a state we may already be compromised in, because a secret sitting in a code repository has to be treated as already exposed the moment it's committed, and an all-access permission model means we have no way of limiting the damage even if we catch something small next week. Fixing these first isn't just the highest-severity item — it's the one where every day of delay is a day the exposure is live. Everything else on this list makes us more resilient going forward; these two need to happen this week.